

Experiment 10

Python BDD and Behave Framework Setup

This experiment sets up a Python behaviour-driven development (BDD) test for the SauceDemo login flow. The goal is to express the requirement in plain language while connecting each sentence to Selenium automation that can be executed from VS Code.

Problem statement

Automation scripts often expose implementation details that are difficult for business analysts and product owners to review. A BDD approach reduces that gap by describing expected behaviour in readable Gherkin scenarios and keeping the technical Selenium code behind those steps.

Objective

Build, run, and verify an end-to-end login scenario using Python, Behave, Gherkin, and Selenium WebDriver.

Tools and concepts

Item	Purpose
Python 3	Runs the automation code.
Behave	Executes Gherkin scenarios and maps them to Python steps.
Selenium WebDriver	Controls Chrome and verifies the application.
Gherkin	Describes behaviour with Feature, Scenario, Given, When, Then, and And keywords.
VS Code	Provides the project editor and integrated terminal.

Implementation

Create a project folder named saucedemo-bdd. Inside it, create a features folder, a steps subfolder, and the files supplied with the accompanying project. The feature file describes a successful login without embedding browser implementation details:

Feature: SauceDemo login

As a shopper

I want to sign in to SauceDemo

So that I can reach the product inventory

Scenario: A registered user signs in successfully

Given the user opens the SauceDemo login page

When the user submits valid SauceDemo credentials

And the user selects the Login button

Then the inventory dashboard is displayed

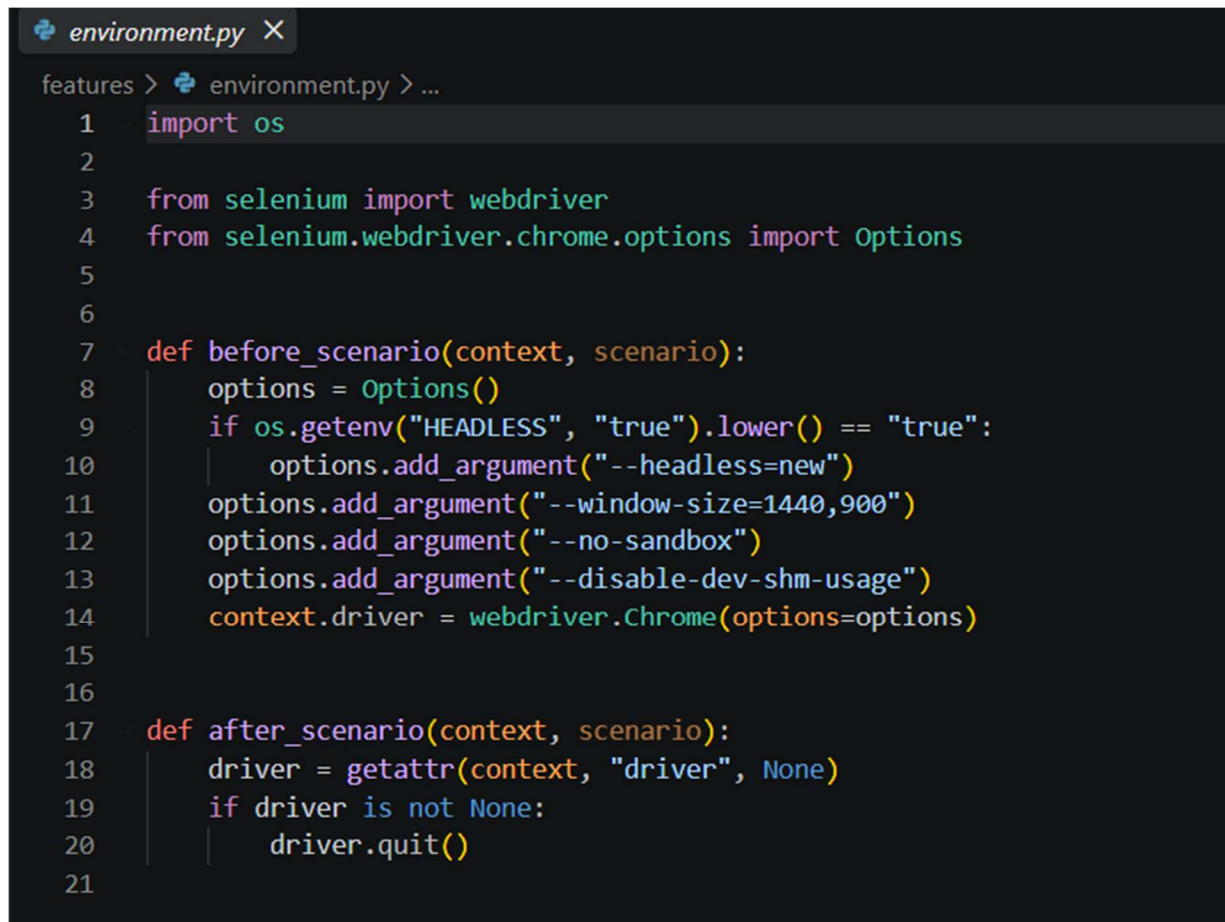
The Python step definitions create a Chrome driver before each scenario, open the site, enter the sample credentials, click Login, wait for the inventory URL, and confirm that the Products heading is present. The driver is closed after every scenario so the test leaves no browser session running.

How to execute

1. Open the saucedemo-bdd folder in VS Code.
2. Create and activate a virtual environment.
3. Install the packages from requirements.txt.
4. Run behave in the integrated terminal.

```
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
behave
```

The default configuration runs Chrome headlessly. Set HEADLESS=false before running Behave if you want to watch the browser. Selenium Manager handles the compatible ChromeDriver when Chrome is installed.



```
environment.py X
features > environment.py > ...
1  import os
2
3  from selenium import webdriver
4  from selenium.webdriver.chrome.options import Options
5
6
7  def before_scenario(context, scenario):
8      options = Options()
9      if os.getenv("HEADLESS", "true").lower() == "true":
10         options.add_argument("--headless=new")
11         options.add_argument("--window-size=1440,900")
12         options.add_argument("--no-sandbox")
13         options.add_argument("--disable-dev-shm-usage")
14         context.driver = webdriver.Chrome(options=options)
15
16
17  def after_scenario(context, scenario):
18      driver = getattr(context, "driver", None)
19      if driver is not None:
20         driver.quit()
21
```

```

PS C:\Users\rahul\Documents\Codex\2026-09-22\r\saucedemo-bdd> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\rahul\Documents\Codex\2026-09-22\r\saucedemo-bdd\.venv\Scripts\Activate.ps1)
(.venv) PS C:\Users\rahul\Documents\Codex\2026-09-22\r\saucedemo-bdd> .\.venv\Scripts\Activate.ps1
>> behave
Feature: SauceDemo login # features/login.feature:1
  As a shopper
  I want to sign in to SauceDemo
  So that I can reach the product inventory
  Scenario: A registered user signs in successfully # features/login.feature:6
    Given the user opens the SauceDemo login page # features/steps/login_steps.py:7
    When the user submits valid SauceDemo credentials # features/steps/login_steps.py:12
    And the user selects the Login button # features/steps/login_steps.py:21
    Then the inventory dashboard is displayed # features/steps/login_steps.py:26

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped, 0 undefined
Took 0m0.778s
(.venv) PS C:\Users\rahul\Documents\Codex\2026-09-22\r\saucedemo-bdd>

```

Expected result

A successful run should report one passed feature, one passed scenario, and four passed steps. If the browser, network, or Chrome installation is unavailable, Behave will report the environmental failure instead of silently marking the scenario as successful.

Observation and conclusion

Behave provides a shared context object for the scenario, allowing the browser instance to be reused by each step. This separation makes the feature file useful as living documentation: a non-developer can read what the test proves, while the step definitions contain the details required to prove it.